

LAPORAN PRAKTIKUM ASD



Muhammad Saum Saifulloh

254107060155

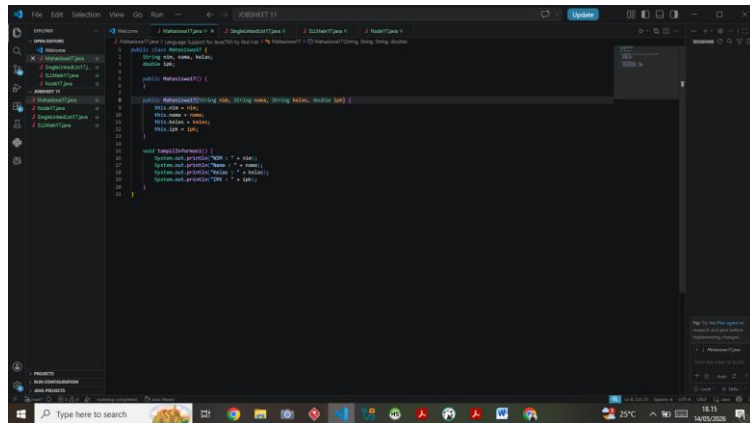
Sistem Informasi Bisnis / 1E

JOBSHEET 11 Linked List

Tujuan Praktikum

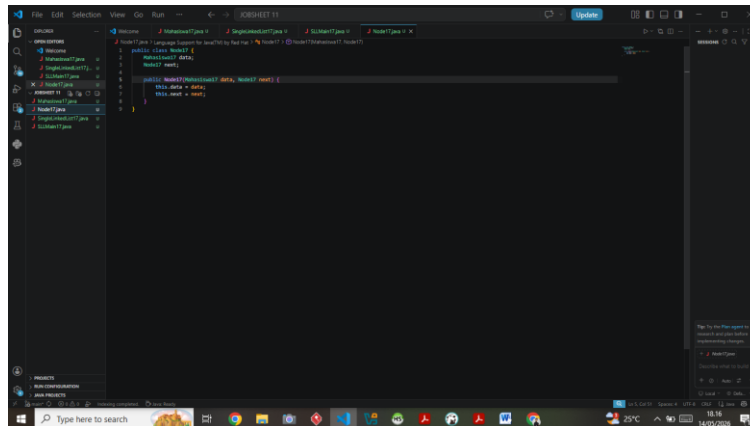
1. Membuat struktur data linked list.
2. Membuat linked list pada program.
3. Membedakan permasalahan yang dapat diselesaikan menggunakan linked list.

Praktikum 2.1 Pembuatan Single Linked List Class Mahasiswa



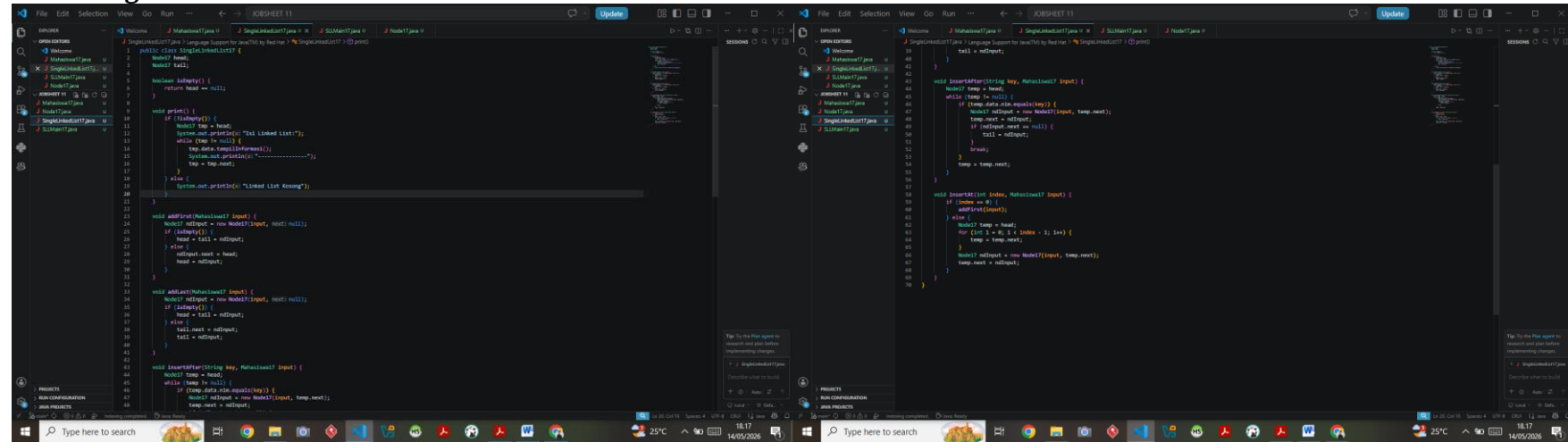
```
1 public class Mahasiswa {
2     String nama;
3     int id;
4     String alamat;
5
6     public Mahasiswa() {
7
8     }
9
10    public Mahasiswa(String nama, String id, String alamat) {
11        this.nama = nama;
12        this.id = id;
13        this.alamat = alamat;
14    }
15
16    void tampilData() {
17        System.out.println("Nama : " + nama);
18        System.out.println("ID : " + id);
19        System.out.println("Alamat : " + alamat);
20    }
21 }
```

Class Node



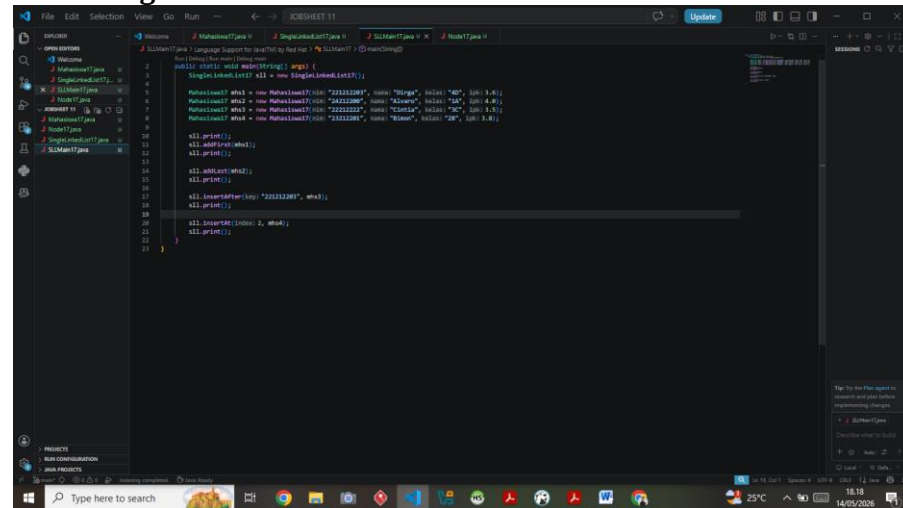
```
1 public class Node {
2     Mahasiswa data;
3     Node next;
4
5     public Node(Mahasiswa data, Node next) {
6         this.data = data;
7         this.next = next;
8     }
9 }
```

Class SingleLinkedList



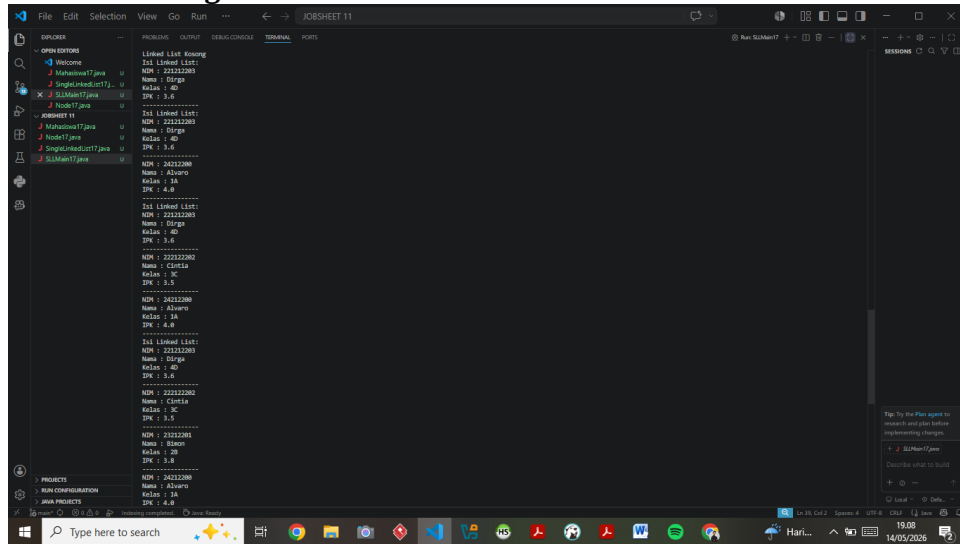
```
1 public class SingleLinkedList {
2     Node head;
3     Node tail;
4
5     boolean isEmpty() {
6         return head == null;
7     }
8
9     void print() {
10        if (isEmpty()) {
11            System.out.println("Not Linked List");
12            return;
13        }
14        while (temp != null) {
15            System.out.print(temp.data + " ");
16            temp = temp.next;
17        }
18        System.out.println("Linked List Ending");
19    }
20
21    void addFirst(Node input) {
22        Node temp = head;
23        if (isEmpty()) {
24            head = tail = input;
25        } else {
26            input.next = head;
27            head = input;
28        }
29    }
30
31    void addLast(Node input) {
32        Node temp = head;
33        if (isEmpty()) {
34            head = tail = input;
35        } else {
36            temp.next = input;
37            tail = input;
38        }
39    }
40
41    void insertAfter(String key, Node input) {
42        Node temp = head;
43        while (temp != null) {
44            if (temp.data.equals(key)) {
45                input.next = temp.next;
46                temp.next = input;
47            }
48            temp = temp.next;
49        }
50    }
51
52    void deleteFirst() {
53        head = head.next;
54    }
55
56    void deleteLast() {
57        if (head == null) return;
58        Node temp = head;
59        while (temp.next != null) {
60            temp = temp.next;
61        }
62        temp.next = null;
63    }
64
65    void deleteKey(String key) {
66        Node temp = head;
67        while (temp != null) {
68            if (temp.data.equals(key)) {
69                if (temp == head) {
70                    head = temp.next;
71                } else {
72                    temp.next = temp.next.next;
73                }
74            }
75            temp = temp.next;
76        }
77    }
78
79    void insertAtIndex(int index, Node input) {
80        if (index == 0) {
81            addFirst(input);
82        } else if (index == 1) {
83            addLast(input);
84        } else {
85            Node temp = head;
86            for (int i = 0; i < index - 1; i++) {
87                temp = temp.next;
88            }
89            input.next = temp.next;
90            temp.next = input;
91        }
92    }
93
94    void printIndex(int index) {
95        Node temp = head;
96        for (int i = 0; i < index; i++) {
97            temp = temp.next;
98        }
99        System.out.println(temp.data);
100    }
101}
```

Class SingleLinkedListMain



```
1 public class SingleLinkedListMain {
2     public static void main(String[] args) {
3         SingleLinkedList list = new SingleLinkedList();
4
5         list.addFirst(10);
6         list.addFirst(20);
7         list.addFirst(30);
8         list.addFirst(40);
9         list.addFirst(50);
10        list.addFirst(60);
11        list.addFirst(70);
12        list.addFirst(80);
13        list.addFirst(90);
14        list.addFirst(100);
15
16        list.print();
17
18        list.deleteFirst();
19        list.print();
20
21        list.deleteLast();
22        list.print();
23
24        list.insertAfter(30, 15);
25        list.print();
26
27        list.insertAtIndex(2, 15);
28        list.print();
29
30        list.deleteKey(30);
31        list.print();
32    }
33}
```

Hasil Run Program



```
Linked List Kosong
Isi Linked List:
NIM : 221212083
Name : Gings
Kelas : 4D
SPK : 3.6
-----
Isi Linked List:
NIM : 221212080
Name : Alvaro
Kelas : 1A
SPK : 4.8
-----
Isi Linked List:
NIM : 221212083
Name : Gings
Kelas : 4D
SPK : 3.6
-----
Isi Linked List:
NIM : 221212082
Name : Cintia
Kelas : 3C
SPK : 3.5
-----
Isi Linked List:
NIM : 221212081
Name : Alvaro
Kelas : 1A
SPK : 4.8
-----
Isi Linked List:
NIM : 221212083
Name : Gings
Kelas : 4D
SPK : 3.6
-----
Isi Linked List:
NIM : 221212085
Name : Cintia
Kelas : 3C
SPK : 3.5
-----
Isi Linked List:
NIM : 221212085
Name : Simon
Kelas : 1A
SPK : 3.8
-----
Isi Linked List:
NIM : 241212080
Name : Alvaro
Kelas : 1A
SPK : 4.8
```

Jawaban Pertanyaan 2.1.2

1. Mengapa output awal “Linked List Kosong”?

Karena objek linked list baru dibuat dan belum memiliki node sehingga `head == null`.

2. Kegunaan variabel temp

Variabel `temp` digunakan sebagai pointer sementara untuk traversal node tanpa mengubah `head`.

3. Modifikasi input keyboard

Dapat menggunakan class `Scanner` agar data mahasiswa diinput user.

Analisis

Pada praktikum ini dibuat struktur data Single Linked List menggunakan class `Node` dan `Mahasiswa`.

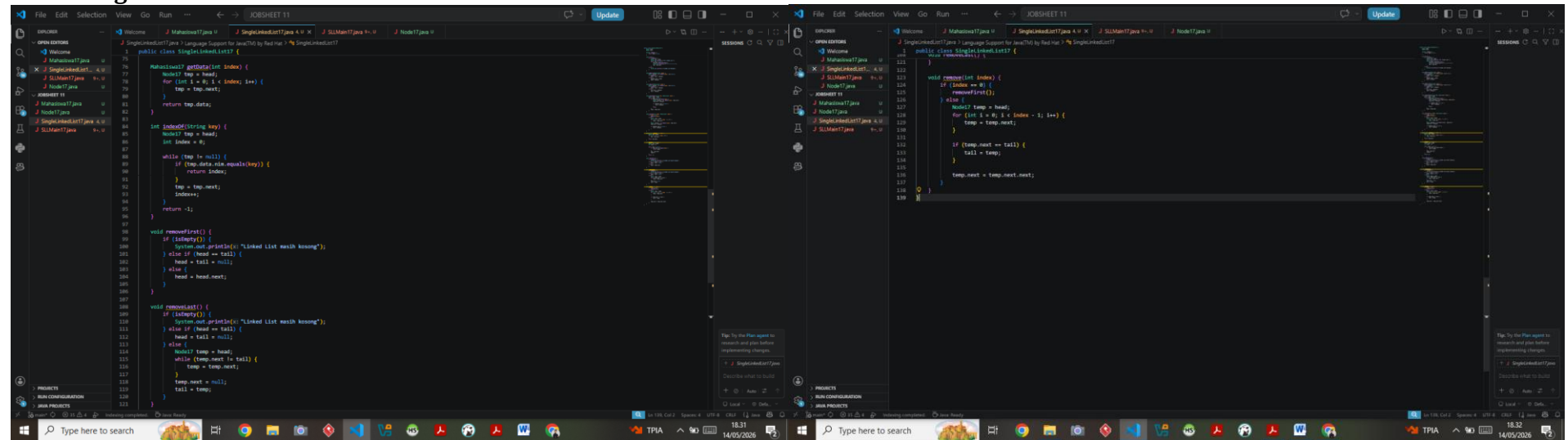
Class `Node00` digunakan untuk menyimpan data mahasiswa dan pointer ke node berikutnya.

Method:

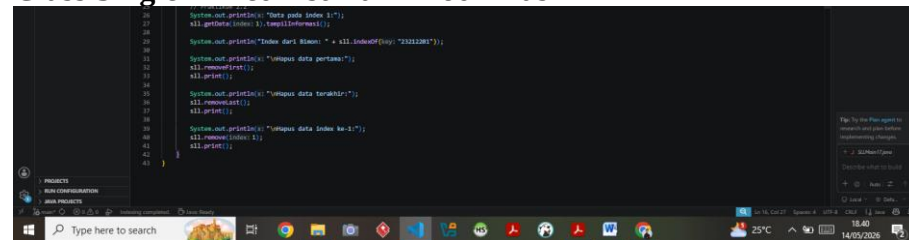
- `addFirst()` → tambah depan
- `addLast()` → tambah belakang
- `insertAfter()` → tambah setelah node tertentu
- `insertAt()` → tambah berdasarkan index
- `print()` → traversal seluruh node
- Linked List memudahkan manipulasi data dinamis dibanding array.

Praktikum 2.2 Modifikasi Pada Elemen Single Linked List

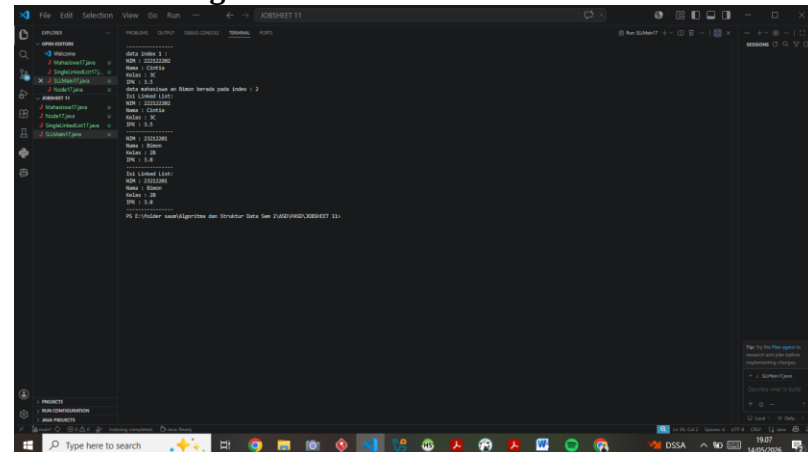
Class SingleLinkedList Modifikasi



Class SingleLinkedListMain Modifikasi



Hasil Run Program



Jawaban Pertanyaan 2.2.3

1. Mengapa memakai break pada remove?

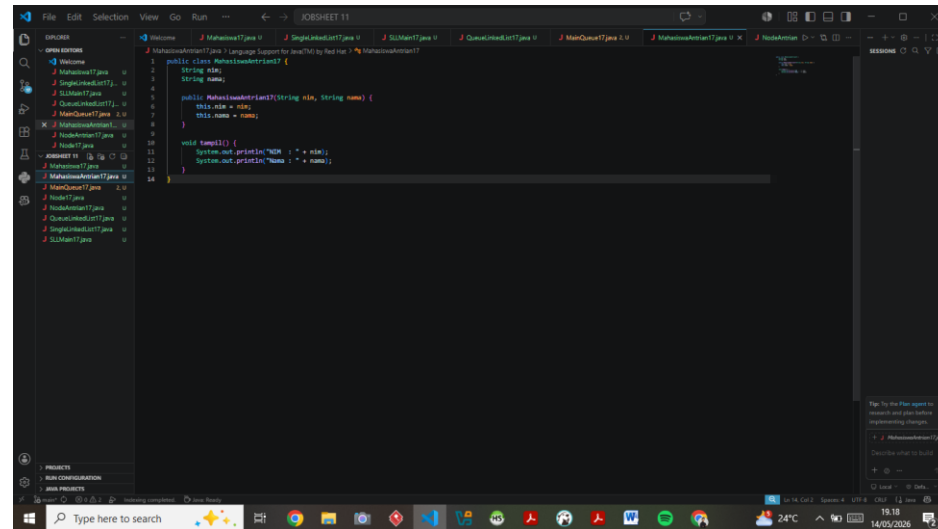
Agar setelah data ditemukan dan dihapus, perulangan berhenti sehingga proses lebih efisien.

2. Kegunaan kode pada method remove

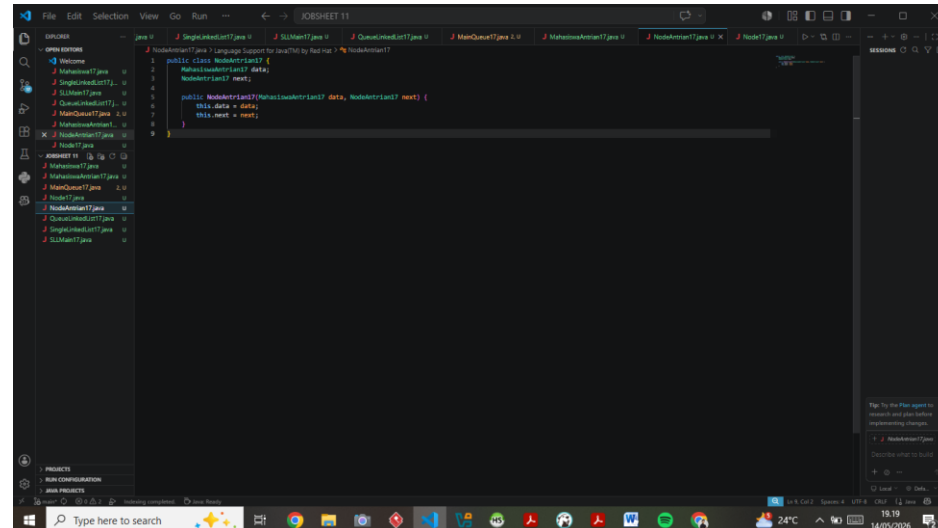
Kode tersebut digunakan untuk menghubungkan node sebelum dan sesudah node yang dihapus agar linked list tetap tersambung.

Tugas Praktikum

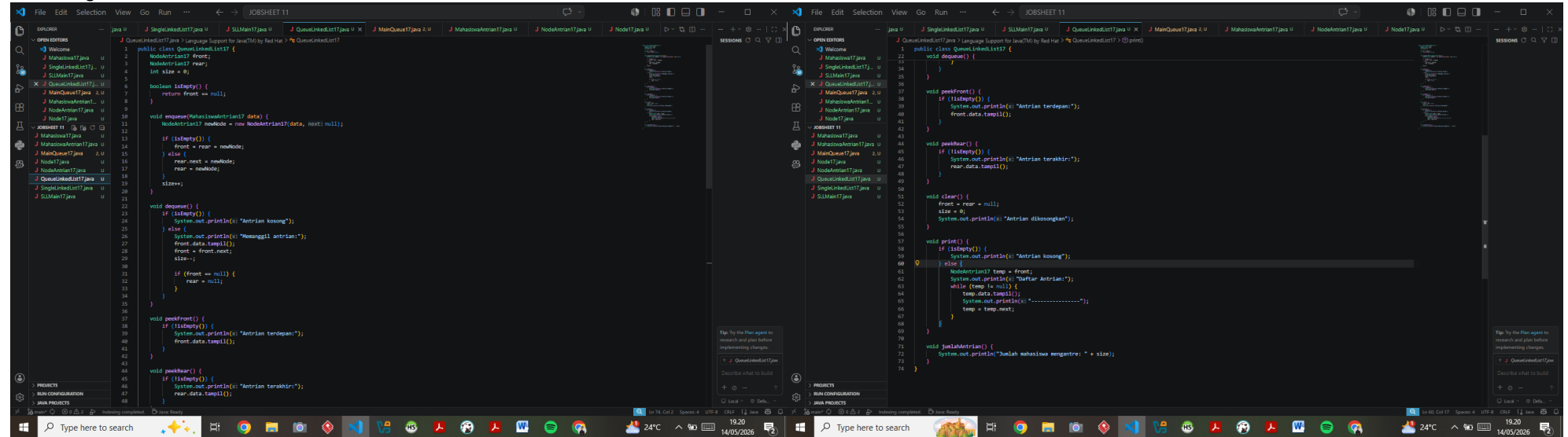
Class MahasiswaAntrian



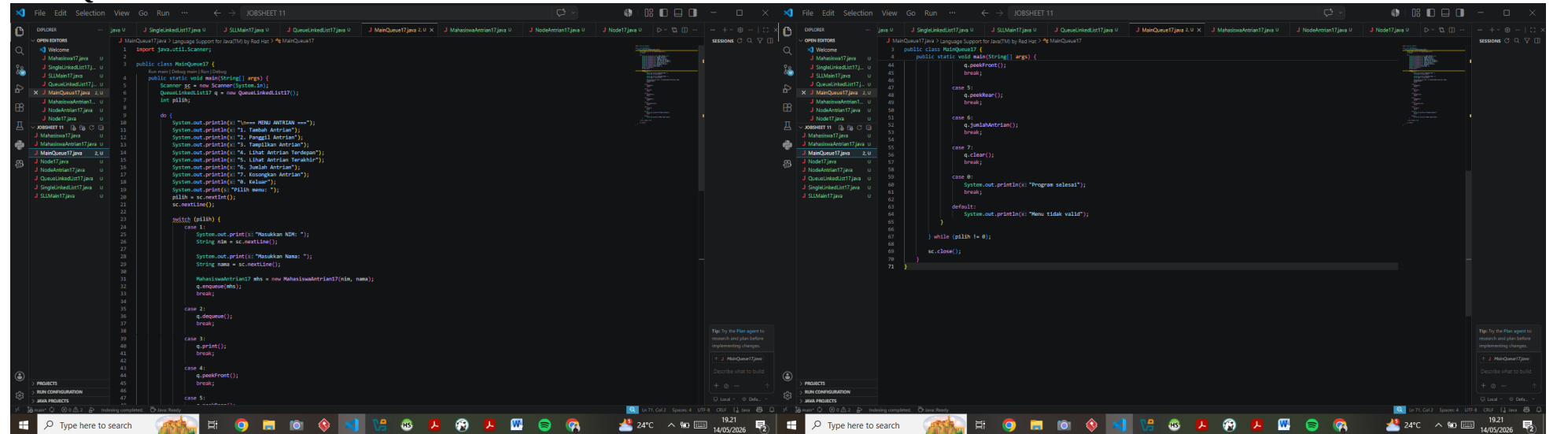
Class NodeAntrian



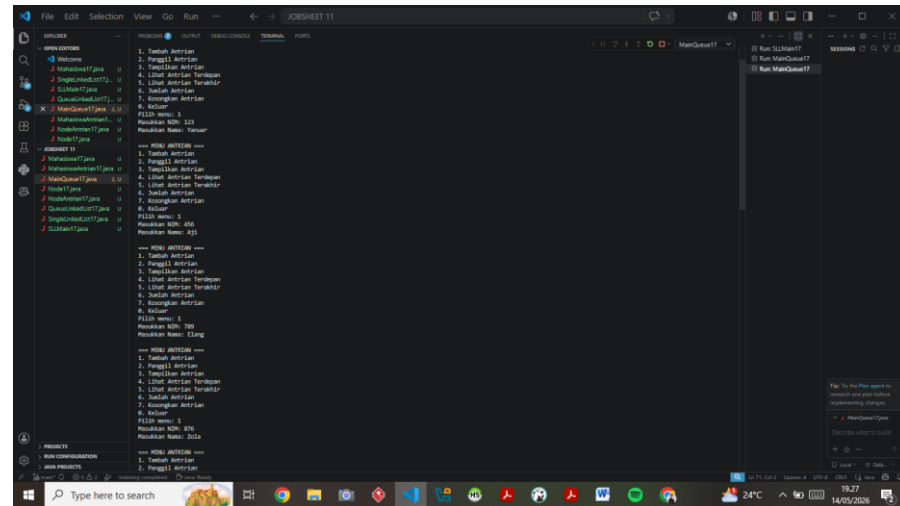
Class QueueLinkedList



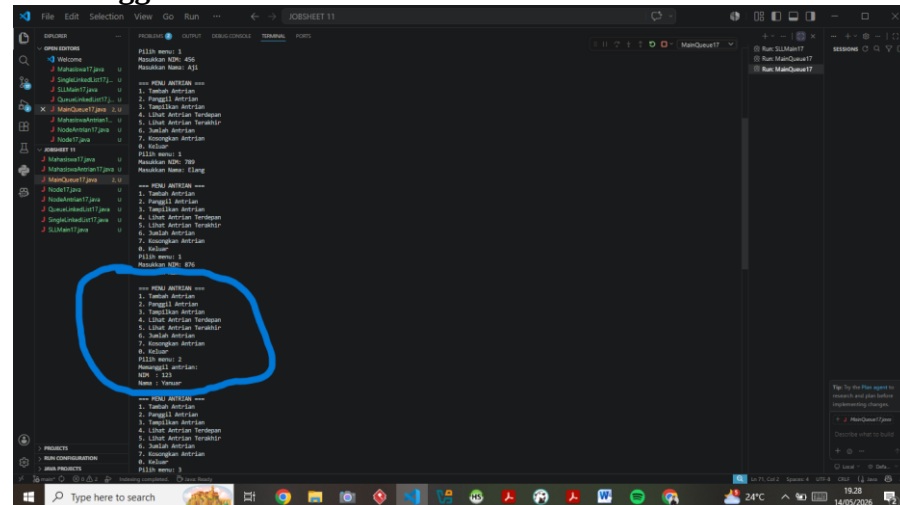
Class QueueMain



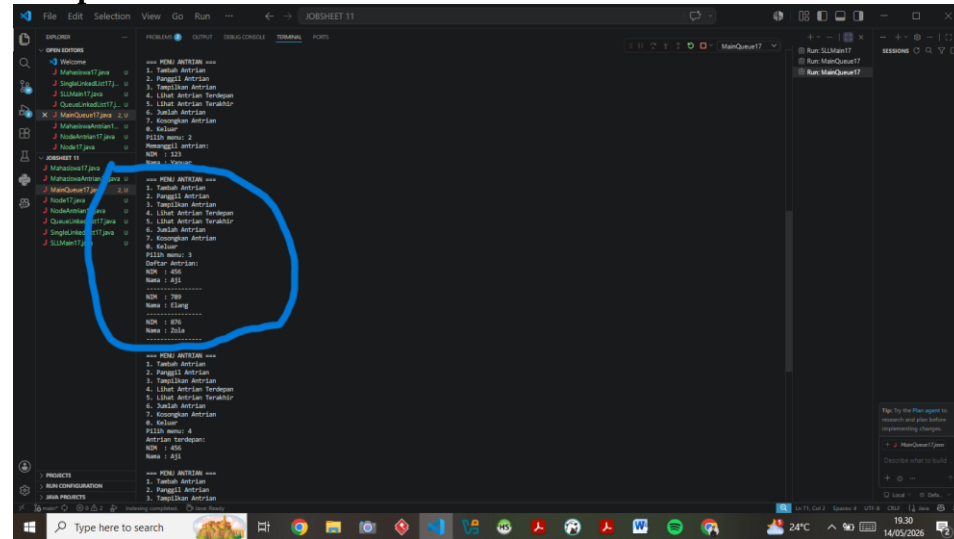
Hasil Run Program Menambahkan Antrian



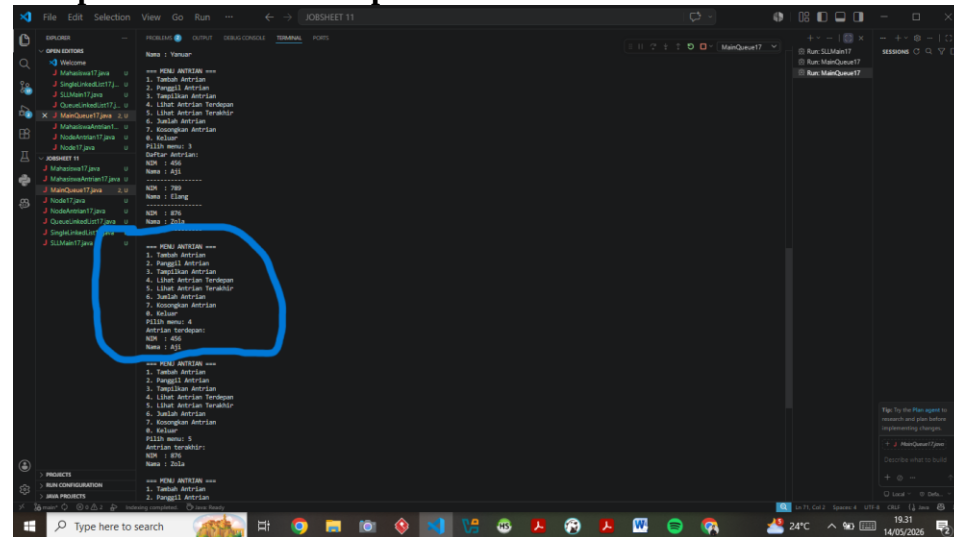
Memanggil Antrian



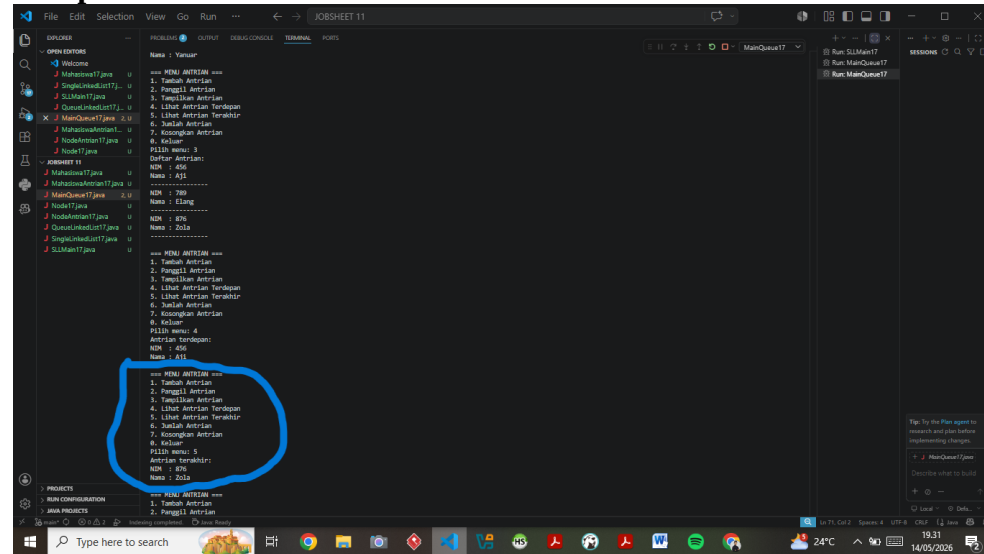
Tampilkan Antrian



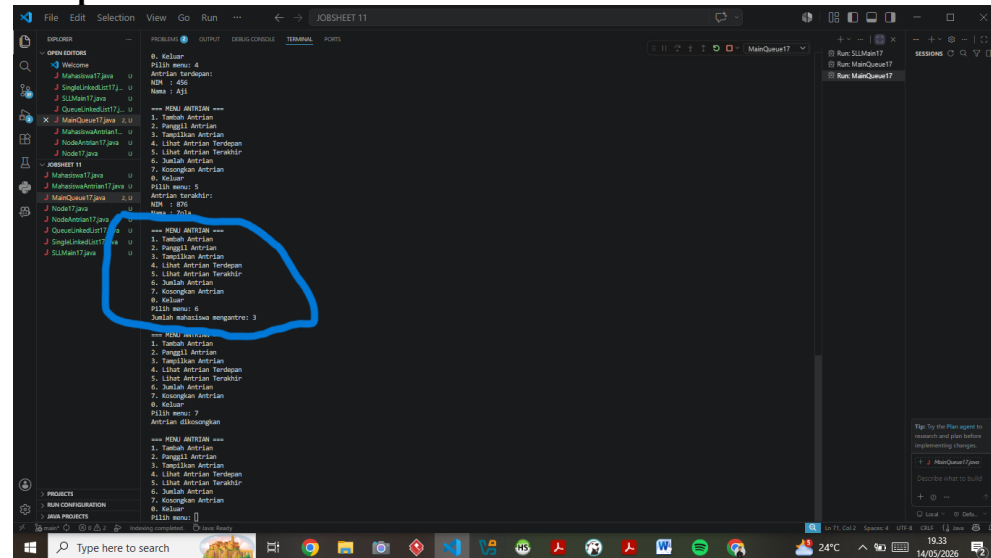
Tampilkan Antrian Terdepan



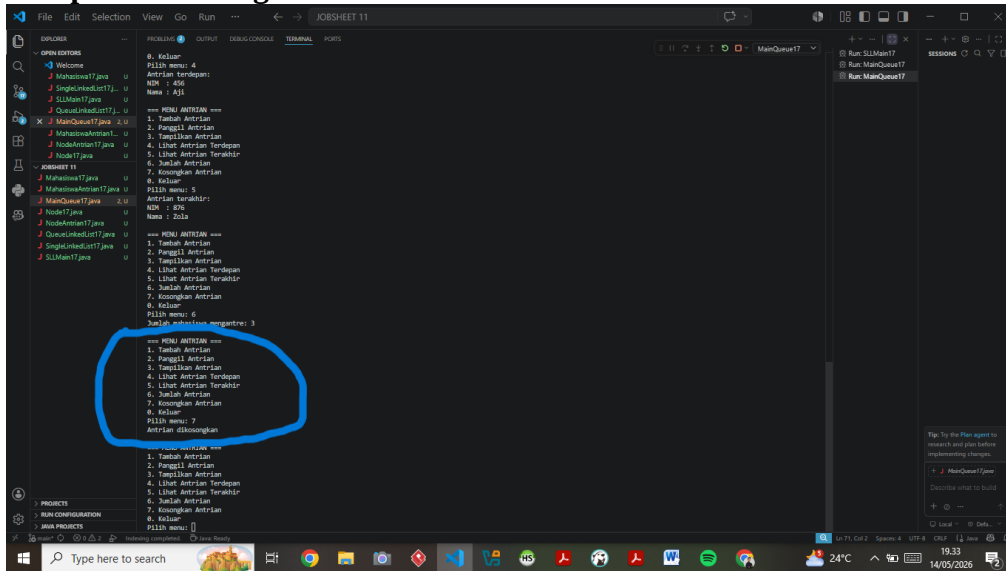
Tampilkan Antrian Terakhir



Tampilkan Jumlah Antrian



Tampilkan Kosongkan Antrian



```
File Edit Selection View Go Run ... JOBSHEET 11
0. Keluar
Pilih menu: 4
Antrian Terdepan:
NDM : 406
Nama : dji

===== MENU ANTRIAN =====
1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Antrian
4. Lihat Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Kosongkan Antrian
0. Keluar
Pilih menu: 5
Antrian terakhir:
NDM : 826
Nama : dila

===== MENU ANTRIAN =====
1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Antrian
4. Lihat Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Kosongkan Antrian
0. Keluar
Pilih menu: 6
Jumlah antrian yang antri: 3

===== MENU ANTRIAN =====
1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Antrian
4. Lihat Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Kosongkan Antrian
0. Keluar
Pilih menu: 7
Antrian dikosongkan

===== MENU ANTRIAN =====
1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Antrian
4. Lihat Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Kosongkan Antrian
0. Keluar
Pilih menu: []
```

Link GitHub: <https://github.com/saumsaifulloh/AlgoritmaDanStrukturDataSem2/tree/main/ASD/PASD/JOBSHEET%2011>